CODE AUDIT REPORT

Frontend Code Review

Cine Rental · React 18 + Vite + Tailwind

Prepared by: Senior Frontend Engineer

Review type: Full codebase audit (read-only)

Date: 23 June 2026

Overall score: 6.5 / 10 (up from 5.5)

Project Overview

Cine Rental is a React single-page application that displays a grid of movies available for rental. Users can open a movie detail modal, add/remove movies to/from a cart (with a live counter and toast notifications), and toggle a dark/light theme. The project was converted from a static HTML template (`html_template/`) into a React + Vite + Tailwind app and has since grown a real state layer.

Stack

Concern	Choice
Framework	React 18 (`StrictMode`)
Build tool	Vite 5
Styling	Tailwind CSS 3 (`darkMode: "class"`)
State	React Context (`MovieContext` , `ThemeContext`)
Prop checking	`prop-types`
Notifications	`react-toastify`

Source layout reviewed

```
src/
├── App.jsx            # context providers + ToastContainer
├── Page.jsx           # layout shell, applies dark class
├── main.jsx
├── Header.jsx         # cart trigger, theme toggle, cart counter
├── Footer.jsx
├── Sidebar.jsx        # data-driven nav
├── index.css
├── cine/
│   ├── MovieList.jsx
│   ├── MovieCard.jsx
│   ├── MovieModal.jsx
│   ├── CartDetail.jsx
│   └── StarRating.jsx
├── context/index.js   # MovieContext + ThemeContext
├── data/movies.js
└── utils/
    ├── cine-utils.js
    └── toast-config.js
```

Overall impression. This codebase has clearly progressed. The cart is now a real, working feature backed by Context; dark mode and toast feedback are implemented; `Sidebar` and `CartDetail` are data-driven with an empty state; and the earlier `prop-types` bug is fixed. That's meaningful growth. What remains is a tighter, well-defined set of issues: a couple of **genuine bugs** (unstable IDs; the modal's "Add to Cart" is a broken anchor with a 404 icon), some **performance hygiene** around Context, and the **accessibility gaps** that were flagged before and are still open (anchors-as-buttons, modals with no keyboard support). None are large; the project is now close to a solid intermediate standard.

Progress Since the Last Review

Credit where due — the following earlier findings were **resolved**:

- ✅ `prop-types` **typo fixed** — `MovieModal` now uses `.isRequired` correctly.
- ✅ **Context implemented** — `MovieContext` / `ThemeContext` have providers in `App.jsx`; the cart is wired end-to-end (add, remove, live counter).
- ✅ `CartDetail` **is data-driven** — maps over `cartData` with a proper empty state and a working Remove button.
- ✅ **Dark mode works** — `darkMode: "class"` in Tailwind + a toggle in `Header`.
- ✅ **Toast feedback** — `react-toastify` now gives user feedback on cart actions.
- ✅ `MovieCard` **cleaned up** — the clickable card is a real `<button>`, the cover has meaningful `alt={movie.title}`, and the tag icon is properly imported.
- ✅ `Sidebar` **refactored** — nav items are mapped from a data array.

Summary of Findings

#	Severity	Area	Issue
1	🔴 High	JS / Bug	Movie IDs generated with `crypto.randomUUID()` at module load — unstable identity
2	🔴 High	React / Bug	`MovieModal` "Add to Cart" is `<a href="#">` (scroll-jump) with a broken `./assets/tag.svg` icon (404)
3	🟠 Medium	Performance	Context value objects not memoized; two unrelated concerns re-render together
4	🟠 Medium	A11y	`<a href="#">` still used as buttons in Header, MovieModal, CartDetail
5	🟠 Medium	A11y / UX	Modals have no Escape, focus trap, backdrop-close, or scroll lock
6	🟠 Medium	React	`MovieCard` keeps redundant `selectedMovie` state and renders one modal per card
7	🟡 Low	Maintainability	Duplicated "Add to Cart" markup has **diverged** between card and modal
8	🟡 Low	Maintainability	`movie` propTypes shape duplicated across two files
9	🟡 Low	Component	`StarRating` renders only filled stars; breaks on non-integers
10	🟡 Low	Architecture	Module-level side effect in `MovieList` (`getAllMovies()` at import)
11	🟡 Low	UX	Dark-mode choice not persisted; resets on reload
12	🟡 Low	UX	"Checkout" button has no handler
13	🔵 Info	Hygiene	Commented-out `console.log`s left in `MovieCard` and `CartDetail`
14	🔵 Info	Perf / A11y	No image dimensions/lazy-loading; theme icon `alt` is always "moon icon"

Legend: 🔴 must fix · 🟠 should fix · 🟡 nice to fix · 🔵 informational

---

Detailed Issues

Issue 1 — Unstable IDs generated at module load

LOCATION

`src/data/movies.js`, the `data` array (each object's `id`).

PROBLEM

Every movie's `id` is computed by calling `crypto.randomUUID()` while the module is evaluated, so IDs are **regenerated on every page load / hot reload**. This is now more consequential than before, because the cart relies on `id` for both de-duplication (`cartData.find(item => item.id === movie.id)`) and removal (`filter(item => item.id !== movie.id)`). It works within a single session (the same in-memory objects are reused), but the moment you persist the cart (localStorage, a backend, a shareable URL) the IDs won't match across reloads, silently breaking "already in cart" detection and removal.

CURRENT CODE

```
const data = [
  {
    id: crypto.randomUUID(),    // ❌ new value on every load
    cover: "once-in-ho.jpg",
    // ...
  },
  // ...
];
```

RECOMMENDED IMPROVEMENT

```
const data = [
  {
    id: "once-upon-a-time-hollywood", // ✅ stable, human-readable slug
    cover: "once-in-ho.jpg",
    // ...
  },
  // ...
];
```

For real data this `id` comes from the backend. For static seed data, hardcode a stable slug or integer.

EXPLANATION

An identifier is *identity* — it must be **stable, unique, and predictable** for the lifetime of the entity. React uses it as a `key`, and your cart logic uses it for equality. Reserve `crypto.randomUUID()` for entities genuinely created at runtime (e.g., a new order line), never for seed data that should be constant.

Issue 2 — Modal "Add to Cart" is a broken anchor with a 404 icon

LOCATION

`src/cine/MovieModal.jsx`, lines 28–35 (the "Add to Cart" `<a>`).

PROBLEM

This single element has **two distinct bugs**:

1. **It's an `<a href="#">`.** The click handler calls `event.stopPropagation()` but never `event.preventDefault()`, so the browser still follows `href="#"` and **scrolls the page to the top** (and pushes `#` into the URL) every time a user adds from the modal.
2. **The icon path is broken.** `src="./assets/tag.svg"` is a hand-written runtime string. Vite doesn't track it, and there's no `public/assets/`, so it resolves relative to the page URL and **404s** — the tag icon is invisible.

Notice that `MovieCard` already fixed both of these (it uses a `<button>` and an imported `TagIcon`), but the modal — duplicated markup — was left behind. This is the DRY risk from the last review materialising.

CURRENT CODE

```
<a
  className="bg-primary rounded-lg ..."
  href="#"
  onClick={(e) => onAddToCart(e, movie)}
>
  <img src="./assets/tag.svg" alt="" />   {/* 404 */}
  <span>${movie.price} | Add to Cart</span>
</a>
```

RECOMMENDED IMPROVEMENT

```
import TagIcon from "../assets/tag.svg";

<button
  type="button"
  className="bg-primary rounded-lg ..."
  onClick={(e) => onAddToCart(e, movie)}
>
  <img src={TagIcon} alt="" />
  <span>${movie.price} | Add to Cart</span>
</button>
```

EXPLANATION

Two lessons. First, **use `<button>` for actions** — it avoids the `href="#"` scroll-jump entirely and is correct for assistive tech (see Issue 4). If you *must* keep an anchor, you have to call `event.preventDefault()`. Second, **import assets so the bundler can resolve and fingerprint them** — never hand-write `./assets/...` strings in JSX. The fact that the card and modal diverged here is exactly why duplicated UI should be extracted into one shared component (Issue 7).

Issue 3 — Context values not memoized; unrelated concerns coupled

LOCATION

`src/App.jsx`, `App` component (the two `Provider` `value` props).

PROBLEM

Each render of `App` creates **brand-new objects** for both context values (`{ cartData, setCartData }` and `{ darkMode, setDarkMode }`). Because the object *identity* changes every render, every component that consumes the context re-renders even if the data it cares about didn't change. Concretely: toggling **dark mode** re-renders every `MovieContext` consumer (all cards), and adding to the **cart** re-renders the theme consumer — the two concerns are coupled through the same parent. On this small app it's invisible, but it's the seed of real performance problems as the tree grows.

CURRENT CODE

```
function App() {
  const [cartData, setCartData] = useState([]);
  const [darkMode, setDarkMode] = useState(true);
  return (
    <ThemeContext.Provider value={{ darkMode, setDarkMode }}>
      <MovieContext.Provider value={{ cartData, setCartData }}>
        <Page />
        <ToastContainer />
      </MovieContext.Provider>
    </ThemeContext.Provider>
  );
}
```

RECOMMENDED IMPROVEMENT

```
const themeValue = useMemo(() => ({ darkMode, setDarkMode }), [darkMode]);
const cartValue  = useMemo(() => ({ cartData, setCartData }), [cartData]);

return (
  <ThemeContext.Provider value={themeValue}>
    <MovieContext.Provider value={cartValue}>
      <Page />
      <ToastContainer />
    </MovieContext.Provider>
  </ThemeContext.Provider>
);
```

Even better, move each piece of state into its own small provider component (`<CartProvider>`, `<ThemeProvider>`) so each owns and memoizes its own value.

EXPLANATION

Context propagates by **reference identity**: consumers re-render when the `value` object changes identity, not when its contents change. Wrapping the value in `useMemo` keyed on the underlying state keeps the reference stable until the data actually changes. Splitting unrelated state into separate providers is the deeper fix — it keeps theme changes from touching cart consumers and vice versa.

Issue 4 — Anchors used as buttons

LOCATION

`src/Header.jsx` (notification, theme toggle, cart trigger), `src/cine/MovieModal.jsx` (Add to Cart, Cancel), `src/cine/CartDetail.jsx` (Checkout, Cancel), `src/Sidebar.jsx` (nav items).

PROBLEM

Controls that perform an in-page action are written as `<a href="#" onClick>`. This is wrong both semantically and practically: clicking an `href="#"` link **jumps the page to the top**, and screen readers announce these as *links* ("navigates somewhere") rather than *buttons* ("performs an action"), misleading assistive-tech users. `MovieCard` was already corrected to use `<button>`; the remaining controls should follow.

CURRENT CODE

```
// Header.jsx — theme toggle
<a onClick={() => setDarkMode(d => !d)} href="#">
  <img src={darkMode ? Sun : Moon} alt="moon icon" />
</a>

// MovieModal.jsx — Cancel
<a onClick={onClose} href="#">Cancel</a>
```

RECOMMENDED IMPROVEMENT

```
<button type="button" onClick={() => setDarkMode(d => !d)}
        aria-label="Toggle dark mode">
  <img src={darkMode ? Sun : Moon} alt="" />
</button>

<button type="button" onClick={onClose}>Cancel</button>
```

Keep real `<a href="...">` only for genuine navigation (the logo; future routes). The Sidebar items are arguably future navigation links, so they may stay anchors — but with real `href`s, not `#`.

EXPLANATION

HTML semantics underpin accessibility: **`<button>` performs an action; `<a href>` navigates.** A native `<button>` is keyboard-focusable, fires on Enter/Space, and is announced correctly — all for free — and it avoids the `href="#"` scroll-jump. When an icon's meaning is already conveyed by an `aria-label` or adjacent text, set the image `alt=""` so it isn't announced twice.

Issue 5 — Modals lack keyboard, focus, and scroll handling

LOCATION

`src/cine/MovieModal.jsx` and `src/cine/CartDetail.jsx`.

PROBLEM

Both modals render a full-screen overlay but implement none of the standard dialog behaviours: **no Escape-to-close**, **no focus trap** (keyboard focus stays on the page behind the overlay, and never returns to the trigger), **clicking the backdrop does nothing**, **background scroll isn't locked**, and there's **no ARIA** (`role="dialog"` , `aria-modal` , labelled title). For mouse users it mostly works; for keyboard and screen-reader users it's effectively a trap.

CURRENT CODE

```
<div className="fixed top-0 left-0 w-screen h-screen z-50 bg-black/60 ...">
  <div className="absolute left-1/2 top-1/2 ...">
    {/* content; only a Cancel link closes it */}
  </div>
</div>
```

RECOMMENDED IMPROVEMENT

```
useEffect(() => {
  const onKey = (e) => e.key === "Escape" && onClose();
  document.addEventListener("keydown", onKey);
  document.body.style.overflow = "hidden";       // lock scroll
  return () => {                                   // cleanup on unmount
    document.removeEventListener("keydown", onKey);
    document.body.style.overflow = "";
  };
}, [onClose]);

return (
  <div className="fixed inset-0 z-50 bg-black/60 ..." onClick={onClose}
       role="dialog" aria-modal="true" aria-labelledby="movie-title">
    <div onClick={(e) => e.stopPropagation()}>...</div>
  </div>
);
```

For production, a primitive like **Radix UI Dialog** or **Headless UI Dialog** handles focus trapping and ARIA correctly out of the box.

EXPLANATION

A dialog is one of the hardest accessibility patterns to get right, which is why the **WAI-ARIA Authoring Practices** specify it precisely (focus enters the dialog, is trapped while open, returns to the trigger on close; Escape closes). This is also the textbook use of `useEffect` : subscribe to a global event and a side effect (scroll lock) on mount, and **clean both up** on unmount via the returned function — forgetting that cleanup is a classic leak / duplicate-listener bug.

Issue 6 — Redundant `selectedMovie` state; one modal per card

LOCATION

`src/cine/MovieCard.jsx` , `MovieCard` component (`isOpen` , `selectedMovie`).

PROBLEM

Each `MovieCard` owns `isOpen` and `selectedMovie` state and renders its own `<MovieModal>`. `selectedMovie` is *always* set to this card's own `movie`, so it duplicates a value the component already has as a prop — two state variables doing the job of one boolean. With 5 movies there are also 5 latent modal instances in the tree. Harmless at this size, but it scatters "which modal is open" across N components instead of one source of truth.

CURRENT CODE

```
const [isOpen, setIsOpen] = useState(false);
const [selectedMovie, setSelectedMovie] = useState(null);

function handleMovieSelected() {
  setSelectedMovie(movie);  // always === movie
  setIsOpen(true);
}
```

RECOMMENDED IMPROVEMENT

```
const [isOpen, setIsOpen] = useState(false);
// open: setIsOpen(true); close: setIsOpen(false);
// {isOpen && <MovieModal movie={movie} onClose={() => setIsOpen(false)} ... />}
```

At scale, lift selection to `MovieList` (or Context) and render **one** modal driven by a single `selectedMovie`.

EXPLANATION

A core React principle: **don't store in state what you can derive from props.** `selectedMovie` duplicates `movie`, creating two things to keep in sync for no benefit. "Which item is open" is a single piece of UI state and belongs in one place — typically the parent that owns the list.

Issue 7 — Duplicated "Add to Cart" markup has diverged

LOCATION

`MovieCard.jsx` (lines 57–64) and `MovieModal.jsx` (lines 28–35).

PROBLEM

The "Add to Cart" control is written twice. They have now **drifted apart**: the card uses a `<button>` with an imported `TagIcon`, while the modal uses an `<a href="#">` with a broken `./assets/tag.svg` path (Issue 2). This is the exact failure mode duplication invites — a fix applied in one copy and missed in the other.

CURRENT CODE

```
// MovieCard.jsx — correct
<button onClick={(e) => handleAddToCart(e, movie)} ...>
  <img src={TagIcon} alt="" /> <span>${movie.price} | Add to Cart</span>
</button>

// MovieModal.jsx — diverged & broken
<a href="#" onClick={(e) => onAddToCart(e, movie)} ...>
  <img src="./assets/tag.svg" alt="" /> <span>${movie.price} | Add to Cart</span>
</a>
```

RECOMMENDED IMPROVEMENT

```
// src/cine/AddToCartButton.jsx
import TagIcon from "../assets/tag.svg";
export default function AddToCartButton({ movie, onAddToCart, className }) {
  return (
    <button type="button" className={className} onClick={(e) => onAddToCart(e, movie)}>
      <img src={TagIcon} alt="" />
      <span>${movie.price} | Add to Cart</span>
    </button>
  );
}
// used by BOTH MovieCard and MovieModal
```

EXPLANATION

DRY ("Don't Repeat Yourself") isn't about typing less — it's about having **one place to change behaviour**. Extracting the shared control into a single component guarantees the card and modal can't diverge, and the bug in Issue 2 simply couldn't have happened.

Issue 8 — Duplicated `movie` propTypes shape

LOCATION

`MovieCard.jsx` and `MovieModal.jsx` (identical `PropTypes.shape({ ... })`).

PROBLEM

The full `movie` shape is copy-pasted in both files. Two copies means two places to update and room to drift — the same root cause as Issue 7.

CURRENT CODE

```
// Repeated verbatim in both files
movie: PropTypes.shape({
  id: PropTypes.string.isRequired,
  cover: PropTypes.string.isRequired,
  title: PropTypes.string.isRequired,
  description: PropTypes.string.isRequired,
  genre: PropTypes.string.isRequired,
  rating: PropTypes.number.isRequired,
  price: PropTypes.number.isRequired,
}).isRequired
```

RECOMMENDED IMPROVEMENT

```
// src/types/movie.js
import PropTypes from "prop-types";
export const moviePropType = PropTypes.shape({
  id: PropTypes.string.isRequired,
  cover: PropTypes.string.isRequired,
  title: PropTypes.string.isRequired,
  description: PropTypes.string.isRequired,
  genre: PropTypes.string.isRequired,
  rating: PropTypes.number.isRequired,
  price: PropTypes.number.isRequired,
});

// usage
MovieCard.propTypes = { movie: moviePropType.isRequired };
```

EXPLANATION

Shared shapes belong in **one canonical definition**. Centralising the `movie` shape (and eventually replacing prop-types with a TypeScript `Movie` type) means a field change is made once and every consumer stays correct.

Issue 9 — `StarRating` renders only filled stars

LOCATION

`src/cine/StarRating.jsx`, `StarRating` component.

PROBLEM

`Array(value).fill(Star)` produces exactly `value` filled stars and nothing else — no fixed scale and no empty stars, so a 5-star and a 3-star movie render rows of different lengths with no visual baseline. It also assumes `value` is a non-negative integer: a decimal like `4.5` makes `Array(4.5)` throw a `RangeError`. Using the array index as `key` is acceptable only because the list is static.

CURRENT CODE

```
function StarRating({ value }) {
  const stars = Array(value).fill(Star);
  return (
    <>
      {stars.map((star, index) => (
        <img key={index} src={star} width="14" height="14" alt="star icon" />
      ))}
    </>
  );
}
```

RECOMMENDED IMPROVEMENT

```
function StarRating({ value, max = 5 }) {
  const filled = Math.round(value);
  return (
    <div role="img" aria-label={`Rating: ${value} out of ${max}`}>
      {Array.from({ length: max }, (_, i) => (
        <img key={i} src={i < filled ? StarFilled : StarEmpty}
             width="14" height="14" alt="" />
      ))}
    </div>
  );
}
```

EXPLANATION

A rating widget should render a **fixed scale** so the proportion is obvious, and be **robust to unexpected input** (decimals, zero, undefined). A single `role="img"` + `aria-label` is also far better for screen readers than five repeated "star icon" announcements.

Issue 10 — Module-level side effect in `MovieList`

LOCATION

`src/cine/MovieList.jsx`, line 5.

PROBLEM

`getAllMovies()` runs at **module scope**, so data is produced as a side effect of *importing* the file rather than of rendering the component. With static data it's benign, but it's the wrong shape: when `getAllMovies` becomes an async API call it can't express loading/error states and runs before React is involved.

CURRENT CODE

```
import { getAllMovies } from "../data/movies";
import MovieCard from "./MovieCard";

const movies = getAllMovies();   // ❌ runs at import time

function MovieList() { return (/* maps movies */); }
```

RECOMMENDED IMPROVEMENT

```
function MovieList() {
  const movies = getAllMovies();      // sync, inside render
  // — or, when async —
  // const { data, isLoading, error } = useMovies();
  return (/* ... */);
}
```

EXPLANATION

Data access should be owned by the **component lifecycle**, not the module system — that's what lets React represent loading/error/empty states and keeps imports free of side effects.

Issue 11 — Dark-mode preference is not persisted

LOCATION

`src/App.jsx` (`darkMode` state), `src/Page.jsx` (applies the class).

PROBLEM

The theme toggle works, but the choice lives only in component state, so it **resets to the default on every reload** and doesn't respect the user's OS preference. Applying `dark` to a wrapper `<div>` (rather than `<html>`) also works here only because everything — including the `fixed` modals — is nested inside it; it's slightly more fragile than toggling the root element.

CURRENT CODE

```
const [darkMode, setDarkMode] = useState(true);   // always starts dark
// Page.jsx
<div className={`h-full w-full ${darkMode ? "dark" : ""}`}>
```

RECOMMENDED IMPROVEMENT

```
const [darkMode, setDarkMode] = useState(
  () => localStorage.getItem("theme")
    ? localStorage.getItem("theme") === "dark"
    : window.matchMedia("(prefers-color-scheme: dark)").matches
);

useEffect(() => {
  document.documentElement.classList.toggle("dark", darkMode);
  localStorage.setItem("theme", darkMode ? "dark" : "light");
}, [darkMode]);
```

EXPLANATION

A theme is a **user preference**, and preferences should survive reloads (`localStorage`) and default to the system setting (`prefers-color-scheme`). The lazy `useState(() => ...)` initializer reads storage once on mount instead of on every render. Toggling the class on `document.documentElement` (`<html>`) is the convention Tailwind documents.

Issue 12 — "Checkout" button does nothing

LOCATION

`src/cine/CartDetail.jsx` , lines 57–68 (Checkout `<a>`).

PROBLEM

The Checkout control is an `<a href="#">` with no `onClick` and no destination. It looks actionable but does nothing — the same "promise to the user that isn't kept" as a dead toggle. Combined with Issue 4, it also scroll-jumps.

RECOMMENDED IMPROVEMENT

At minimum disable it or wire a handler:

```
<button type="button" disabled={cartData.length === 0}
        onClick={handleCheckout}>
  <img src={CheckoutIcon} alt="" /> <span>Checkout</span>
</button>
```

EXPLANATION

Interactive affordances should either **do something or not appear interactive**. A button that looks live but is inert erodes user trust and confuses testers.

Issue 13 — Commented-out logging left in source

LOCATION

`src/cine/MovieCard.jsx` line 15, `src/cine/CartDetail.jsx` line 15.

PROBLEM

`// console.log("Cart data in MovieCard", cartData)` and a similar line in `CartDetail` are dead, commented-out debugging statements. They add noise, hint at unfinished debugging, and tend to accumulate.

RECOMMENDED IMPROVEMENT

Delete them. If you need diagnostics in development, prefer a guarded logger (`if (import.meta.env.DEV) console.log( ... )`) or your browser's debugger.

EXPLANATION

Source control is your history — you don't need commented-out code "just in case." Clean code communicates intent; leftover debug lines communicate the opposite. A linter rule (`no-console`) or a pre-commit hook can enforce this.

Issue 14 — Minor performance & accessibility polish

LOCATION

`MovieCard.jsx` / `MovieModal.jsx` (images); `Header.jsx` (theme icon alt).

PROBLEM

- Movie cover `<img>` s have **no** `width` / `height` (or `aspect-ratio`), so the browser can't reserve space before load → **layout shift (CLS)**.
- Images aren't **lazy-loaded**; all covers fetch eagerly.
- The theme-toggle icon always has `alt="moon icon"` even when the sun icon is shown, and the notification bell is a non-functional `<a href="#">` .

RECOMMENDED IMPROVEMENT

```
<img className="w-full object-cover aspect-[2/3]"
     src={getUrlImage(movie.cover)}
     width={300} height={450} loading="lazy"
     alt={`${movie.title} poster`} />

<img src={darkMode ? Sun : Moon} alt="" />  {/* label lives on the button */}
```

EXPLANATION

Cumulative Layout Shift is a Core Web Vital; giving images intrinsic dimensions lets the browser reserve the box so content doesn't jump. `loading="lazy"` defers off-screen images for free. And `alt` text must match what is actually shown — stale alt text is worse than none.

Overall Code Review Score

⭐ 6.5 / 10 *(up from 5.5)*

Dimension	Score	Notes
Architecture	6/10	Context layer added and providers wired; still flat-ish structure and a module-level side effect.
React practices	6/10	Real working state + good `MovieCard` refactor; redundant state and unmemoized context remain.
Code quality	6/10	Readable and consistent; held back by unstable IDs, a broken asset path, and leftover debug logs.
Performance	6/10	Fine at this size; context memoization, image sizing, and lazy-loading still open.
Maintainability	6/10	Data-driven Sidebar/Cart help; duplicated Add-to-Cart markup already diverged into a bug.
Accessibility/UX	5/10	Improved (semantic card button, empty state), but anchors-as-buttons and modal a11y persist.

Why this score. The project took clear, real steps forward: a functioning cart with Context, dark mode, toasts, an empty state, and several clean refactors. That lifts it out of "scaffold" territory. It's held back from a 7–8 by two genuine bugs (unstable IDs; the modal's broken anchor + 404 icon), a performance footgun (unmemoized, coupled context), and the still-open accessibility work. Crucially, the duplicated "Add to Cart" markup has already *caused* a bug — the strongest possible argument for the DRY fixes below. Fixing the two 🔴 items and memoizing context would put this at a solid 7.5.

Senior Engineer Recommendations

1. Top 5 improvements to make first

1. **Fix the two correctness bugs (Issues 1–2).** Give movies stable IDs, and convert the modal's "Add to Cart" to a `<button>` with an imported icon.
2. **De-duplicate the cart control (Issues 7–8).** Extract one `AddToCartButton` and one `moviePropType`; the modal bug literally can't recur afterwards.
3. **Memoize and split context (Issue 3).** `useMemo` both values, ideally behind `CartProvider` / `ThemeProvider`, so theme and cart stop re-rendering each other.
4. **Finish accessibility (Issues 4–5).** Convert remaining action `<a>` s to `<button>`, and make both modals keyboard-accessible (Escape, focus trap, backdrop close, scroll lock).
5. **Persist the theme and harden `StarRating` (Issues 11, 9).** `localStorage` + `prefers-color-scheme`; fixed star scale robust to decimals.

2. Skills / concepts to learn (mapped to the mistakes)

- **TypeScript** — would catch the unstable-ID type-misuse, the duplicated shape (Issue 8), and prop drift at compile time. Highest-leverage next step.
- **React rendering & Context performance** — reference identity, `useMemo`, splitting providers (Issue 3).
- **Web accessibility (WAI-ARIA Authoring Practices)** — button vs. link, the dialog pattern, alt-text correctness (Issues 4, 5, 14).
- **How Vite handles assets** — `import` vs. `public/` vs. `import.meta.url` (Issue 2).
- **Component composition / DRY** — extracting shared UI to prevent divergence (Issues 7, 8).
- **`useEffect` and cleanup** — subscriptions and side effects with teardown (Issues 5, 11).

3. Production-level practices to adopt

- **TypeScript + strict mode** alongside or instead of prop-types.
- **CI running** `lint` + `build` on every push; the project already has a strict `--max-warnings 0` lint script — wire it into CI and add `no-console`.
- **A headless component library** (Radix / Headless UI) for accessible dialogs.
- **A data-fetching layer** (React Query / SWR) the moment data goes async, with explicit loading/error/empty states.
- **Prettier + a defined folder convention** (`components/` , `features/`) so the codebase stays consistent as it grows.
- **Tests** (Vitest + React Testing Library) for cart add/remove/dedupe logic and `StarRating` edge cases.

4. How this code compares to professional frontend standards

This is now a **competent intermediate React project**. A production codebase would add type safety, memoized/segmented state, accessible-by-default semantics, no dead code, and no silent runtime failures. Cine Rental has the *shape* of good React — working Context, component decomposition, prop validation, user feedback — and is beginning to show the *rigor* (the `MovieCard` and `Sidebar` refactors are exactly the right instincts). The remaining gaps are well-understood, teachable patterns rather than fundamental misunderstandings. Closing the 🔴/🟠 items and adding TypeScript would bring it within reach of professional standard.

Learning Roadmap

1. **Week 1 — Correctness & hygiene:** Fix Issues 1, 2, 13. Run `npm run lint` and clear every warning. Read how Vite resolves static assets.
2. **Week 2 — Composition & DRY:** Extract `AddToCartButton` and `moviePropType` (Issues 7, 8); simplify `MovieCard` 's modal state (Issue 6).
3. **Week 3 — State & performance:** Split and memoize context into `CartProvider` / `ThemeProvider` (Issue 3); persist the theme (Issue 11).
4. **Week 4 — Accessibility:** Convert anchors to buttons; rebuild modals on Radix/Headless UI Dialog; test with keyboard only (Issues 4, 5).
5. **Week 5 — TypeScript:** Migrate the `movie` shape and components to TS and watch the compiler catch these bug classes for you.
6. **Week 6 — Tests & polish:** Vitest + RTL for cart logic and `StarRating` ; image dimensions + lazy loading; wire or disable Checkout (Issues 9, 12, 14).

End of review. This document is advisory; no source files were modified during this audit.